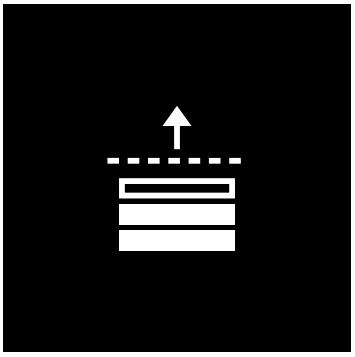


Favo

Arquitetura do banco de dados: SQLite que guarda sessões, histórico de operações, traces e benchmarks do painel.

Banco de dados · versão 1.0.0

Copyright (©) 2026 Erik Pimenta · licença MIT

Sumário

1. Visão geral
2. Diagrama de relacionamentos
3. Tabelas
4. Restrições e índices
5. Migrações
6. Consultas usadas pela API
7. Backup, restauração e manutenção
8. Usando PostgreSQL no lugar do SQLite

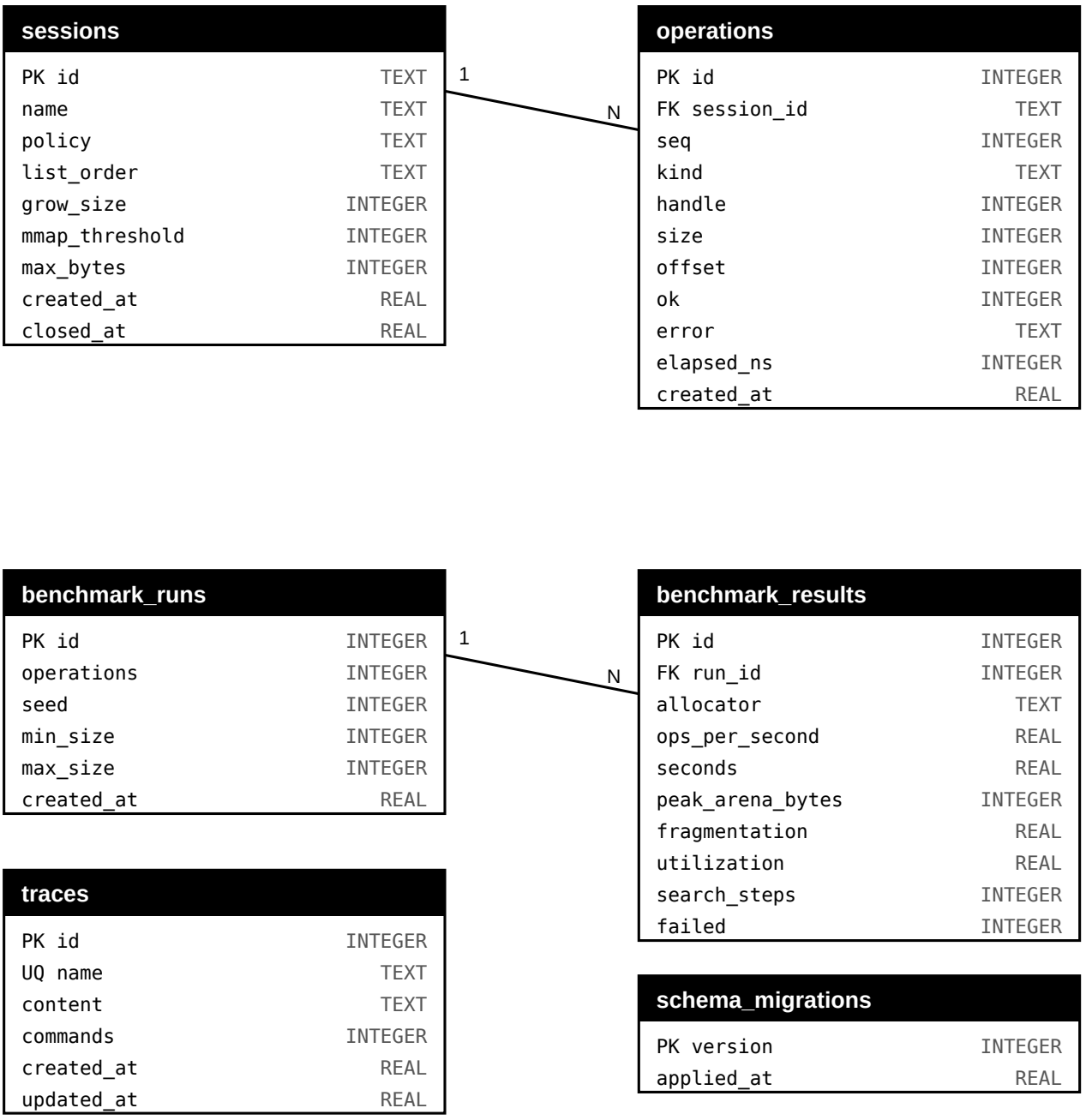
1. Visão geral

O alocador em si não usa banco de dados: tudo o que ele gerencia vive na memória do processo. O banco existe para a API e o painel lembrarem o que aconteceu: quais sessões foram abertas, cada operação feita em cada heap, os traces salvos e os resultados dos benchmarks.

O banco é um **SQLite** em um arquivo só, criado sozinho na primeira execução. Não há servidor para instalar.

Item	Valor
Motor	SQLite 3 (módulo `sqlite3` da biblioteca padrão do Python)
Arquivo	`data/favo.db` (variável `FAVO_DATABASE`); no Docker, `/data/favo.db` no volume `favo-dados`
Modo de diário	WAL (`PRAGMA journal_mode = WAL`): leituras não bloqueiam a escrita
Chaves estrangeiras	Ligadas em toda conexão (`PRAGMA foreign_keys = ON`)
Espera por lock	5 segundos (`PRAGMA busy_timeout = 5000`)
Código	`api/favo_api/store.py`
Datas	Segundos desde 1970 em UTC, tipo REAL (`time.time()`)

2. Diagrama de relacionamentos



Uma sessão tem muitas operações e uma execução de benchmark tem muitos resultados (um por alocador). `traces` e `schema_migrations` são independentes. Apagar uma sessão ou uma execução apaga em cascata as linhas filhas.

3. Tabelas

3.1 sessions

Uma linha por heap criado pela API. A sessão viva fica na memória; a linha serve de histórico.

Campo	Tipo	Regra	Descrição
id	TEXT	PK	12 caracteres hexadecimais aleatórios
name	TEXT	NOT NULL	Nome dado no painel (até 80 caracteres)
policy	TEXT	CHECK first-fit, next-fit, best-fit	Política atual (atualizada na troca)
list_order	TEXT	CHECK address, lifo	Ordem da lista circular
grow_size	INTEGER	> 0	Bytes de cada pedido ao sistema
mmap_threshold	INTEGER	≥ 0	Limite para bloco grande (0 desliga)
max_bytes	INTEGER	≥ 0	Teto de memória da sessão
created_at	REAL	NOT NULL	Criação
closed_at	REAL	nulo enquanto aberta	Fechamento manual ou por inatividade

3.2 operations

Cada chamada que muda o heap de uma sessão, com sucesso ou erro.

Campo	Tipo	Regra	Descrição
id	INTEGER	PK AUTOINCREMENT	Identificador
session_id	TEXT	FK sessions(id) ON DELETE CASCADE	Sessão
seq	INTEGER	UNIQUE (session_id, seq)	Número sequencial dentro da sessão
kind	TEXT	CHECK alloc, calloc, realloc, memalign, free, write, policy, trim, reset, trace	Tipo da operação
handle	INTEGER	nulo	Número do bloco na sessão
size	INTEGER	nulo	Bytes pedidos, gravados ou comandos do trace
offset	INTEGER	nulo	Deslocamento do bloco dentro da arena (o endereço absoluto muda a cada execução)
ok	INTEGER	CHECK 0 ou 1	Se deu certo
error	TEXT	nulo	Mensagem do alocador quando falhou

Campo	Tipo	Regra	Descrição
elapsed_ns	INTEGER	padrão 0	Tempo da operação em nanossegundos
created_at	REAL	NOT NULL	Momento

3.3 traces

Campo	Tipo	Regra	Descrição
id	INTEGER	PK AUTOINCREMENT	Identificador
name	TEXT	UNIQUE	Nome (letras, números, espaço, ponto, hífen)
content	TEXT	NOT NULL	Comandos, um por linha
commands	INTEGER	≥ 0	Quantidade de comandos válidos
created_at	REAL	NOT NULL	Criação
updated_at	REAL	NOT NULL	Última gravação (salvar com o mesmo nome sobrescreve)

3.4 benchmark_runs

Campo	Tipo	Regra	Descrição
id	INTEGER	PK AUTOINCREMENT	Execução
operations	INTEGER	> 0	Operações por alocador
seed	INTEGER	NOT NULL	Semente do gerador
min_size	INTEGER	> 0	Menor bloco
max_size	INTEGER	CHECK max_size ≥ min_size	Maior bloco
created_at	REAL	NOT NULL	Momento

3.5 benchmark_results

Campo	Tipo	Regra	Descrição
id	INTEGER	PK	Identificador
run_id	INTEGER	FK benchmark_runs(id) ON DELETE CASCADE	Execução
allocator	TEXT	UNIQUE (run_id, allocator)	first-fit, next-fit, best-fit ou malloc do sistema
ops_per_second	REAL	NOT NULL	Vazão
seconds	REAL	NOT NULL	Tempo total

Campo	Tipo	Regra	Descrição
peak_arena_bytes	INTEGER	NOT NULL	Pico de memória pedida ao sistema (0 para o malloc do sistema)
fragmentation	REAL	NOT NULL	Fragmentação no meio da carga
utilization	REAL	NOT NULL	Aproveitamento no meio da carga
search_steps	INTEGER	NOT NULL	Nós visitados nas buscas
failed	INTEGER	NOT NULL	Pedidos que falharam

3.6 schema_migrations

Campo	Tipo	Descrição
version	INTEGER PK	Número da migração aplicada
applied_at	REAL	Quando foi aplicada

4. Restrições e índices

Objeto	Definição	Motivo
idx_operations_session	operations(session_id, seq)	Histórico de uma sessão em ordem, usado pelo painel a cada operação
UNIQUE (session_id, seq)	operations	Impede duas operações com o mesmo número na sessão
idx_results_run	benchmark_results(run_id)	Montar uma execução com os resultados
UNIQUE (run_id, allocator)	benchmark_results	Um resultado por alocador em cada execução
UNIQUE name	traces	Salvar com o mesmo nome atualiza (UPSERT)
CHECK	policy, list_order, kind, ok, tamanhos	O banco recusa valores que a API nunca grava

5. Migrações

As migrações ficam na lista `MIGRATIONS` em `store.py`. Ao abrir o banco, a API aplica, dentro de uma transação, cada migração cujo número ainda não está em `schema_migrations`. Para mudar o esquema, acrescente um novo item no fim da lista, nunca edite um item já publicado.

```
MIGRATIONS = [
    """ ... versão 1 ... """,
    """
    ALTER TABLE sessions ADD COLUMN owner TEXT;
    CREATE INDEX IF NOT EXISTS idx_sessions_created ON sessions(created_at);
    """,
]
```

Para incluir um tipo novo em `operations.kind`, o SQLite exige recriar a tabela: crie `operations_nova` com o `CHECK` novo, copie os dados com `INSERT INTO operations_nova SELECT * FROM operations`, apague a antiga e renomeie, tudo na mesma migração.

6. Consultas usadas pela API

```
SELECT * FROM operations WHERE session_id = ? ORDER BY seq DESC LIMIT ?;

INSERT INTO traces (name, content, commands, created_at, updated_at) VALUES (?, ?, ?, ?, ?)
ON CONFLICT(name) DO UPDATE SET content = excluded.content, commands = excluded.commands,
updated_at = excluded.updated_at;

SELECT * FROM benchmark_results WHERE run_id = ? ORDER BY id;

UPDATE sessions SET closed_at = ? WHERE id = ? AND closed_at IS NULL;
```

6.1 Consultas úteis para análise

```
SELECT kind, COUNT(*) AS total, SUM(ok = 0) AS falhas, AVG(elapsed_ns) / 1000.0 AS media_us
FROM operations GROUP BY kind ORDER BY total DESC;

SELECT r.allocator, AVG(r.ops_per_second) AS ops, AVG(r.fragmentation) AS fragmentacao
FROM benchmark_results r JOIN benchmark_runs b ON b.id = r.run_id
WHERE b.min_size = 16 AND b.max_size = 4096 GROUP BY r.allocator;

SELECT s.name, s.policy, COUNT(o.id) AS operacoes
FROM sessions s LEFT JOIN operations o ON o.session_id = s.id
GROUP BY s.id ORDER BY s.created_at DESC;
```

7. Backup, restauração e manutenção

7.1 Backup com a API rodando

```
sqlite3 data/favo.db ".backup 'favo-backup.db'"
docker compose exec favo python -c "import sqlite3; s=sqlite3.connect('/data/favo.db');
d=sqlite3.connect('/data/backup.db'); s.backup(d)"
docker compose cp favo:/data/backup.db ./favo-backup.db
```

7.2 Restauração

```
docker compose down
docker compose run --rm -v "$PWD:/restore" favo cp /restore/favo-backup.db /data/favo.db
docker compose up -d
```

7.3 Limpeza

```
DELETE FROM sessions WHERE closed_at < strftime('%s', 'now', '-30 days');
DELETE FROM benchmark_runs WHERE created_at < strftime('%s', 'now', '-90 days');
VACUUM;
```

As operações e os resultados saem junto por causa do `ON DELETE CASCADE`.

8. Usando PostgreSQL no lugar do SQLite

Só vale a pena se várias instâncias da API precisarem compartilhar o histórico. O esquema equivalente:

```
CREATE TABLE sessions (
  id TEXT PRIMARY KEY, name TEXT NOT NULL,
  policy TEXT NOT NULL CHECK (policy IN ('first-fit','next-fit','best-fit')),
  list_order TEXT NOT NULL CHECK (list_order IN ('address','lifo')),
  grow_size BIGINT NOT NULL CHECK (grow_size > 0), mmap_threshold BIGINT NOT NULL CHECK
(mmap_threshold >= 0),
  max_bytes BIGINT NOT NULL CHECK (max_bytes >= 0),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(), closed_at TIMESTAMPTZ);
CREATE TABLE operations (
  id BIGSERIAL PRIMARY KEY, session_id TEXT NOT NULL REFERENCES sessions(id) ON DELETE
CASCADE,
  seq INTEGER NOT NULL, kind TEXT NOT NULL, handle INTEGER, size BIGINT, "offset" BIGINT,
  ok BOOLEAN NOT NULL, error TEXT, elapsed_ns BIGINT NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(), UNIQUE (session_id, seq));
CREATE TABLE traces (id BIGSERIAL PRIMARY KEY, name TEXT NOT NULL UNIQUE, content TEXT NOT
NULL,
  commands INTEGER NOT NULL, created_at TIMESTAMPTZ NOT NULL DEFAULT now(), updated_at
TIMESTAMPTZ NOT NULL DEFAULT now());
CREATE TABLE benchmark_runs (id BIGSERIAL PRIMARY KEY, operations BIGINT NOT NULL, seed
BIGINT NOT NULL,
  min_size INTEGER NOT NULL, max_size INTEGER NOT NULL, created_at TIMESTAMPTZ NOT NULL
DEFAULT now());
CREATE TABLE benchmark_results (id BIGSERIAL PRIMARY KEY,
  run_id BIGINT NOT NULL REFERENCES benchmark_runs(id) ON DELETE CASCADE, allocator TEXT NOT
NULL,
  ops_per_second DOUBLE PRECISION NOT NULL, seconds DOUBLE PRECISION NOT NULL,
peak_arena_bytes BIGINT NOT NULL,
  fragmentation DOUBLE PRECISION NOT NULL, utilization DOUBLE PRECISION NOT NULL,
  search_steps BIGINT NOT NULL, failed BIGINT NOT NULL, UNIQUE (run_id, allocator));
```

Na API, troque o `sqlite3` da classe `Store` por `psycopg`, use `%s` no lugar de `?` nos parâmetros e `RETURNING id` no lugar de `lastrowid`. O resto do código não muda, porque todas as consultas passam pela classe `Store`.